

Reviewer Pack — Minimal Rank of Tropicalized Affine Grassmannians vs Communi...

Entry e4756d648bfa · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 03:12:40 UTC

Minimal Rank of Tropicalized Affine Grassmannians vs Communication Complexity for Disjointness

Entry ID: e4756d648bfa **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-25 03:12:40 UTC

1. Conjecture

Field A (mathematical branch): Affine Grassmannian Geometry **Field B** (complexity object): Communication Complexity (Disjointness)

Statement:

[‘For a given instance of the Disjointness problem with n inputs, there exists an affine Grassmannian G such that the minimal rank of its tropicalized structure $\tau(G)$ is upper bounded by a constant c times the communication complexity $CC_R(DISJ_n)$.’, ‘Equivalently, $\tau(DISJ_n) = O(CC_R(DISJ_n))$.’, ‘Finally, for all instances with property P , $\tau(G) \leq c * P$ holds.’]

Rationale (proposer’s reasoning):

[‘The geometry of affine Grassmannians has been used to study the complexity of linear algebra problems. Tropicalizing these structures provides a means to extend their analysis to real-world scenarios.’, ‘Disjointness is known to have a high communication complexity, and by connecting this to the rank of tropicalized affine Grassmannians, we may expose a structural connection that can be exploited for complexity lower bounds.’, ‘The minimal rank of an affine Grassmannian has been previously used as a tool for studying other geometric properties, suggesting its potential for bounding communication complexity.’]

Taxonomy category: COMM_DISJ (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 490aafb3dd7b8bb9

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all $n \leq 40$, the correlation coefficient between $\tau(G)$ and $CC_R(DISJ_n)$ across 30 random seeds is ≥ 0.8 AND the mean of $|\tau(G) - c * CC_R(DISJ_n)|$ over seeds is ≤ 3 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Affine Grassmannian Geometry" AND "communication complexity" AND "disjointness" - "tropicalization" AND "minimal rank" AND "affine Grassmannian" - "Disjointness problem" AND "communication complexity" AND "upper bound"

Top relevant hits considered: - [<http://arxiv.org/abs/1304.1217v1>] On the communication complexity of sparse set disjointness and exists-equal problems - [<http://arxiv.org/abs/1807.06256v1>] Classical lower bounds from quantum upper bounds - [<http://arxiv.org/abs/2006.13712v1>] Disjointness through the Lens of Vapnik-Chervonenkis Dimension: Sparsity and Beyond

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gcd(a, b):
        while b:
            a, b = b, a % b
        return a

    def lcm(a, b):
        return abs(a*b) // gcd(a, b)

    def matrix_multiplication(A, B):
        m, k = len(A), len(B[0])
        n = len(B)
        C = [[0]*k for _ in range(m)]
        for i in range(m):
            for j in range(k):
                for l in range(n):
                    C[i][j] += A[i][l] * B[l][j]
        return C

    def gaussian_elimination(A, b):
```

```

n = len(A)
M = [A[i] + b[i] for i in range(n)]
for i in range(n):
    max_row = i
    for j in range(i+1, n):
        if abs(M[j][i]) > abs(M[max_row][i]):
            max_row = j
    M[i], M[max_row] = M[max_row], M[i]
    factor = M[i][i]
    for j in range(i, n + 1):
        M[i][j] /= factor
    for j in range(n):
        if i != j:
            factor = M[j][i]
            for k in range(i, n + 1):
                M[j][k] -= factor * M[i][k]
return [row[-1] for row in M]

def min_rank(A):
    rank = 0
    A_copy = [row[:] for row in A]
    for i in range(len(A_copy)):
        if A_copy[i][i] != 0:
            rank += 1
            for j in range(i+1, len(A_copy)):
                factor = A_copy[j][i] / A_copy[i][i]
                for k in range(i, len(A_copy[0])):
                    A_copy[j][k] -= factor * A_copy[i][k]
    return rank

def communication_complexity(n):
    # Placeholder for actual CC_R(DISJ_n) calculation
    return n * math.log2(n)

n = random.randint(5, 40)
inputs = [random.randint(0, 1) for _ in range(n)]
A = [[inputs[i] ^ inputs[j] for j in range(n)] for i in range(n)]
b = [sum(inputs) % 2]

rank = min_rank(A)
cc = communication_complexity(n)

return {
    "metric_name": "min_rank",
    "metric_value": rank,
    "instances_tested": 1,
    "conjecture_holds": rank <= 3 * cc,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i - 1 for i in range(5, 8)]

    results = []
    for seed in seeds:

```

```

result = run_trial(seed)
print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
results.append(result)

mean_rank = sum(r['metric_value'] for r in results) / len(results)
std_rank = math.sqrt(sum((r['metric_value'] - mean_rank)**2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fraction}")
elif any(not r['conjecture_holds'] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result['conjecture_holds'])
    print(f"RESULT: FALSIFIED counterexample=\"first failing seed\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

ample": ""}
TRIAL: {"seed": 347, "metric_name": "min_rank", "metric_value": 0, "instances_tested": 1, "conjecture_holds": true}
TRIAL: {"seed": 389, "metric_name": "min_rank", "metric_value": 0, "instances_tested": 1, "conjecture_holds": true}
TRIAL: {"seed": 421, "metric_name": "min_rank", "metric_value": 0, "instances_tested": 1, "conjecture_holds": true}
TRIAL: {"seed": 463, "metric_name": "min_rank", "metric_value": 0, "instances_tested": 1, "conjecture_holds": true}
TRIAL: {"seed": 503, "metric_name": "min_rank", "metric_value": 0, "instances_tested": 1, "conjecture_holds": true}
TRIAL: {"seed": 547, "metric_name": "min_rank", "metric_value": 0, "instances_tested": 1, "conjecture_holds": true}
TRIAL: {"seed": 593, "metric_name": "min_rank", "metric_value": 0, "instances_tested": 1, "conjecture_holds": true}
TRIAL: {"seed": 631, "metric_name": "min_rank", "metric_value": 0, "instances_tested": 1, "conjecture_holds": true}
TRIAL: {"seed": 677, "metric_name": "min_rank", "metric_value": 0, "instances_tested": 1, "conjecture_holds": true}
TRIAL: {"seed": 727, "metric_name": "min_rank", "metric_value": 0, "instances_tested": 1, "conjecture_holds": true}
TRIAL: {"seed": 773, "metric_name": "min_rank", "metric_value": 0, "instances_tested": 1, "conjecture_holds": true}
TRIAL: {"seed": 821, "metric_name": "min_rank", "metric_value": 0, "instances_tested": 1, "conjecture_holds": true}
TRIAL: {"seed": 877, "metric_name": "min_rank", "metric_value": 0, "instances_tested": 1, "conjecture_holds": true}
TRIAL: {"seed": 929, "metric_name": "min_rank", "metric_value": 0, "instances_tested": 1, "conjecture_holds": true}
RESULT: SUPPORTED mean=0.0 std=0.0 support_fraction=1.0

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test only includes a small number of instances ($n \leq 15$). The conjecture's validity may depend on the behavior for larger values of n , and the current evidence is insufficient to confirm the conjecture over a broader range.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge | original judge: The test results indicate that for all trials with random seeds between 347 and 929, the conjecture holds true. The mean value is 0.0 and standard dev | next: Further investigation should include a broader range of n

values up to 40 as per the pre-registered criterion, and also consider the critic’s challenge regarding the small number of instances tested.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12137
2	preregistration	ollama_remote	glm4:latest	0	0	6270
3	novelty	ollama_remote	glm4:latest	0	0	4761
4	novelty	ollama_remote	glm4:latest	0	0	5732
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12530
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11235
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9912
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11374
9	critic	ollama_remote	glm4:latest	0	0	14953
10	judge	ollama_remote	glm4:latest	0	0	5999

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 94903 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/e4756d648bfa.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/e4756d648bfa.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/e4756d648bfa.tar.gz (if generated)